



Instituto Tecnológico y de Estudios Superiores de Monterrey

Seguimiento de actividades de reto

Firmas solidarias:

Tecnología criptográfica para los derechos humanos

Equipo 2:

Valeria Arciga Valencia	- A01737555
Henning Arvid Ladewig	- A01831983
Alberto Boughton Reyes	- A01178500
Ximena Montes Bautista	- A01737949
Diego Octavio Arias Incháustegui	- A00838285
Fernando Daniel Saucedo Hernández	- A01385490

Curso: Uso de álgebras modernas para seguridad y criptografía.
Grupo 603.

Fecha de entrega: 10 de abril de 2026

Índice

Descripción del reto.....	3
Primera aplicación: Sistema de Gestión de Identidades.....	4
Aproximación tentativa.....	5
Herramientas que pueden implementarse.....	8
Uso de herramientas en modo exploratorio:.....	9
Propuesta inicial.....	13
Enlace al repositorio.....	13
Cronograma de Gantt.....	13
Preguntas formuladas por integrante en la sesión con Socio Formador.....	15
Referencias.....	16

Descripción del reto

La migración forzada es un fenómeno global donde millones de personas se ven obligadas a abandonar sus lugares de origen en busca de seguridad, estabilidad, y mejores oportunidades, ya sea por conflictos, crisis económicas, o desastres naturales. En este contexto, Casa Monarca, como Organización Socio Formadora (OSF) dentro del curso de Uso de álgebras modernas para seguridad y criptografía, desempeña un papel esencial brindando apoyo a migrantes en situación de vulnerabilidad mediante servicios críticos como asesoría legal, ayuda humanitaria, orientación laboral, y apoyo educativo. El trabajo de estas organizaciones implica el manejo de información altamente sensible, como datos personales, documentos legales, constancias y registros médicos o jurídicos, información que es de ayuda para la orientación de la población atendida por la OSF.

Actualmente, organizaciones enfrentan el obstáculo de implementación de sistemas de seguridad informática avanzados debido a sus limitaciones de recursos, técnica, dejando información vulnerable. Sin embargo, la legislación mexicana exige a toda organización el manejo y protección adecuada de los datos personales. El problema principal a resolver es brindar herramientas que permitan garantizar que los procesos digitales de Casa Monarca cumplan con modelos como Confidencialidad, Integridad y Disponibilidad, logrando beneficiar sus operaciones.

El proyecto busca brindar a Casa Monarca herramientas tecnológicas robustas que aseguren el cumplimiento de normativas como la Ley Federal de Protección de Datos Personales en Posesión de Particulares (LFPDPPP) y facilitar el ejercicio de Derechos ARCO (Acceso, Rectificación, Cancelación y Oposición) por parte de los beneficiarios de la organización. Todo esto se debe lograr garantizando veracidad, autenticidad y el no repudio de documentos emitidos, sin complicar la operación diaria de la organización ni requerir de conocimientos técnicos avanzados.

Esto se abordará mediante la creación de un sistema de gestión de identidades basado en certificados digitales para dar de alta, revocar y rastrear accesos entre otras aplicaciones para la gestión de información interna y procesos de autenticación de la información enviada por la Organización Socio Formadora.

Es necesario que la implementación de la tecnología sea transparente para todos los usuarios y colaboradores, en donde se pueda mejorar tiempo de atención y productividad. Además, la solución debe ser fácil de usar, adaptable a los recursos disponibles, y escalable, para que en el futuro pueda integrarse con tecnologías más avanzadas como Blockchain.

Primera aplicación: Sistema de Gestión de Identidades

El objetivo dentro de la solución a Casa Monarca es el desarrollo de un sistema de manejo de identidades basado en certificados digitales. Esta primera aplicación supone el permiso de gestión de ciclos de vida de credenciales de usuarios de forma segura, entendiendo que hay un flujo y cambio constante en los usuarios beneficiados por la Organización Socio Formadora.

En primer lugar, el proceso de alta requiere de registros iniciales estructurados en donde se verifique la identidad de colaboradores y usuarios antes de la generación de claves asimétricas y la emisión de certificados. Al contar con identidades activas, el sistema debe de contemplar mecanismos de revocación para cancelar permisos debido a operaciones o a incidentes técnicos como el compromiso de dispositivos.

Por lo tanto, resulta necesaria la integración de protocolos de estado, como las Listas de Revocación de Certificados (CRL), lo que permite invalidar criptográficamente las credenciales antes de su fecha de expiración en el sistema (Cooper et al., 2008), además de la necesidad de implementación de un esquema de Cifrado Asimétrico Dual, lo que asegura que la información cifrada por un colaborador pueda ser recuperada por la administración en casos de contingencia.

Adicionalmente, el control de acceso se estructurará en cuatro niveles de jerarquía confirmados por la organización:

- 1. Nivel 1 (Administrador).** Acceso total al sistema y recuperación de datos. En el caso de la Organización Socio Formadora, corresponde al líder y al responsable de TI.
- 2. Nivel 2 (Coordinador).** Capacidad de visualización, disponibilización y edición de información de su área.
- 3. Nivel 3 (Operativo).** Permiso exclusivo para visualización y subir información.
- 4. Nivel 4 (Captura).** Acceso restringido a la captura y envío de información.

Por otro lado, cuando la relación con el usuario o colaborador concluye de manera definitiva, el sistema debe de ejecutar una baja, lo que implica la expiración de la identidad dentro del sistema. Y adicionalmente, en el orden de cumplir con la garantía de transparencia y cumplimiento normativo, la aplicación integrará un esquema de rastro que mantendrá auditoría continua sobre el acceso a la información y modificaciones realizadas a los datos.

Para reforzar la seguridad en el acceso de los colaboradores a la aplicación, se integra el uso de Microsoft Authenticator como mecanismo de Autenticación Multifactor (MFA), tomando en cuenta que el personal ya cuenta con esta herramienta en sus dispositivos laborales.

Diagrama de Permisos del Usuario

Nivel de Acceso	Perfil	Leer	Escribir	Borrar	Firmar	Revocar
Nivel 1	Administrador	☒	☒	☒	☒	☒
Nivel 2	Coordinador	☒	☒	☐	☒	☒
Nivel 3	Operativo	☒	☒	☐	☐	☐
Nivel 4	Captura/Voluntario	☐	☒	☐	☐	☐
Nivel 5	Beneficiario	☒	☐	☐	☐	☐

Marco de referencia

El manejo de identidades digitales y control de acceso (IAM) es un desafío abordado a nivel global mediante diversas plataformas consolidadas. Soluciones de código abierto como Keycloak ofrecen arquitecturas robustas para la gestión de usuarios, roles y autenticación multifactor. Estas herramientas están diseñadas para entornos empresariales, proporcionando protocolos estándar que garantizan la confidencialidad en la transmisión de datos.

El funcionamiento de Keycloak se fundamenta en actuar como intermediario entre aplicaciones y usuarios, empleando protocolos estándar como OpenID Connect y SAML 2.0. Su arquitectura permite el Single Sign-On (SSO), permitiendo que el usuario se autentique una sola vez para acceder a múltiples servicios, y gestiona la seguridad mediante el intercambio de JSON Web Tokens (JWT), lo cual garantiza integridad y confidencialidad de la identidad en cada transacción.

Sin embargo, la implementación de estos sistemas, a menudo se requiere de infraestructura dedicada que excede las capacidades de alojamiento compartido que tiene actualmente Casa Monarca. Por este motivo es que la propuesta opta por un desarrollo adaptado al socio formador, en donde el sistema implementará un estándar de seguridad y cifrado para los registros.

Después del análisis de los requerimientos de la organización, se ha determinado que no se establecerán distinciones criptográficas ni esquemas de protección diferenciados de edad, como capas extra para menores o demografía, garantizando que toda la información de la población atendida reciba el mismo nivel máximo de protección criptográfica posible sin sobrecargar el servidor.

Aproximación tentativa

Para la materialización de la arquitectura del primer desarrollo que gestione la identidad, la aproximación tentativa de herramientas y exploración se basará en estándares revisados en literatura del área de ciberseguridad. En el centro del sistema, el resguardo de confidencialidad depende del uso de algoritmos de clave pública, tales como los esquemas de curvas elípticas, que protejan la generación y el intercambio de información confidencial, a manera complementaria las funciones resumen, como el estándar SHA-256, para verificar la integridad de los datos almacenados y en tránsito (Zong, 2025).

Se pretende que todo el ciclo de vida del desarrollo del software sea bajo la metodología sugerida, DevSecOps, la cual garantiza la integración continua de la seguridad desde las fases iniciales del diseño, en donde se puedan mitigar vulnerabilidades a través de herramientas de análisis (Mohammed et al., 2025; Nadipalli, 2023).

Trabajando con estas metodologías, podemos utilizar muchas herramientas utilizadas en investigaciones y sistemas reales para servirnos de referencia en el proyecto. Bibliotecas criptográficas como OpenSSL o frameworks de criptografía en Python son muy usados para generar claves y manejar certificados en algoritmos criptográficos modernos. Estas herramientas nos permiten implementar funciones como generación de pares de claves, firmas digitales y validación de certificados en aplicaciones personalizadas (Cryptography, 2026).

Ahora bien, además de los estándares criptográficos que se explicaron previamente, se necesitará de una arquitectura tecnológica para la implementación del sistema. Esta debe ser capaz de integrar lo relacionado a la generación de identidades de los migrantes, gestión de certificados y control de accesos dentro de un entorno lo suficientemente accesible para colaboradores o voluntarios de Casa Monarca.

Dado que “la identidad digital es la representación única de un sujeto que participa en actividades en línea” (Grassi et al., 2017), es especialmente necesario que, al tratarse de información altamente personal y relevante por razones sociopolíticas, nuestra arquitectura funcione correctamente: de lo contrario, pueden haber varios tipos de “daños e impactos”, como “divulgación no autorizada de información confidencial”, “seguridad personal vulnerada” , “infracciones civiles o penales” (Grassi et al., 2017), etc.

Por lo anterior, nuestra primera aproximación al proyecto sería una aplicación web con arquitectura cliente-servidor en la cual el backend se encargue de la lógica de seguridad y la gestión de identidades, mientras que el frontend se encargue de una interfaz efectiva y sencilla que sirva para la interacción de los usuarios con el sistema.

En este contexto, el acceso a la plataforma debe estructurarse mediante un esquema de verificación explícita en el que no exista confianza implícita basada únicamente en la red o la

ubicación del usuario. Como señalan Zuñiga y Hecht (2023), los sistemas modernos de seguridad requieren mecanismos de autorización dinámicos y granulares que permitan asignar privilegios únicamente a los recursos necesarios y durante periodos de tiempo limitados. Aplicado al sistema de Casa Monarca, esto implica que cada voluntario o colaborador acceda a la información sensible únicamente a través de capas de validación que verifiquen su identidad, rol y contexto de uso en cada sesión.

Para expandir esta primera aproximación, es necesario explicar cada uno de los tres elementos. Comenzado por el backend, el lenguaje de programación Python nos parece la opción más adecuada: además de ser el lenguaje más conocido por el equipo de trabajo, ofrece alta compatibilidad con diversas bibliotecas criptográficas y es usado en el desarrollo de sistemas de seguridad (OpenSSL Project, 2023). Las dos bibliotecas anteriormente mencionadas permiten el uso de funciones que serán esenciales para este proyecto, como la generación de pares de claves, emisión y verificación de certificados, etc. (Cryptography Project, 2023). Dado que son bibliotecas, nos permiten llevar a cabo este proyecto sin necesidad de desarrollar algoritmos desde cero, reduciendo el riesgo de errores de implementación.

Ahora bien, en lo relacionado al almacenamiento de información, serán necesarios mecanismos de control que “limiten el acceso a los recursos de un sistema únicamente a los usuarios autorizados” (Stallings, 2017). Para lograrlo, el sistema se apoyará en una base de datos relacional que permite gestionar de forma integral aspectos como el registro de usuarios, las listas de revocación y los registros de auditoría vinculados al sistema de identidades. En este sentido, MySQL destaca como una solución idónea para este tipo de aplicaciones, gracias a su robusto control de concurrencia y sus mecanismos de seguridad integrada, además de ser parte de los servidores dentro de la organización, lo que es una consideración importante.

Lo anterior es posible debido a que MySQL permite administrar los privilegios de acceso mediante un sistema de cuentas de usuario y roles, poniendo a disposición del desarrollador un modelo de control de acceso flexible y seguro (Oracle, 2024). Gracias a las capacidades de este sistema de gestión de bases de datos, es factible mantener una trazabilidad detallada de las operaciones realizadas mediante el uso de registros de auditoría y controles de cambios, asegurando la integridad y supervisión constante de la información.

Referente a la interfaz de usuario, se propone en equipo el desarrollo de una aplicación web sencilla pero eficaz que permita a los colaboradores o voluntarios de Casa Monarca interactuar con el sistema de gestión de identidades de los migrantes de forma clara y segura. Por el momento, se proponen herramientas muy comúnmente usadas, como CSS, JavaScript o HTML, integradas con marcos de desarrollo web como Flask o Django, que permiten construir aplicaciones basadas en Python, el lenguaje de programación que, como se ha mencionado anteriormente, será usado para el proyecto y facilitar la implementación de sistemas seguros y

escalables. En la sección de “Herramientas que podrían usarse” se explorará a mayor profundidad algunas de las mencionadas previamente.

Por último, se usará para el desarrollo de nuestro proyecto diversas herramientas que incluyen control de versiones, como lo es Git, y plataformas de colaboración como GitHub. Sobre el primero, será usado ya que “es un sistema de control de versiones distribuido diseñado para gestionar todo tipo de proyectos, desde los más pequeños hasta los más grandes, con rapidez y eficiencia” (Chacon & Straub, 2014).

Sobre el segundo, que “con herramientas basadas en inteligencia artificial, pruebas de seguridad integradas y una integración perfecta, presta apoyo a los equipos desde el primer commit hasta el desarrollo empresarial” (Github, 2026), parece que se podrán utilizar para elevar el nivel de las soluciones a diversos contextos, manteniendo la claridad necesaria entre el equipo. Así, con las dos herramientas ya mencionadas, se puede tener un registro de versiones que sirva como repertorio de los cambios realizados en el código, además, claro está, de facilitar el trabajo entre los integrantes del equipo de trabajo, permitiendo así una documentación más organizada y fácil de entender para diversas personas involucradas en el proceso como lo pueden ser profesores, colaboradores de Casa Monarca, entre otros.

Herramientas compatibles

De acuerdo al plan de Host que tiene Casa Monarca, se ha realizado la captura de sus herramientas compatibles que se han considerado para el desarrollo de la solución necesaria.

Categoría	Herramientas y Tecnologías Compatibles	Incompatibilidades Notables
Lenguajes y Frameworks	PHP: 8.0, 8.1, 8.2 Python: 3.6 Frameworks: Laravel (10 y 11), Django (solo hasta 3.2), Ruby on Rails, CakePHP Otros: JavaScript, JSON, FastCGI, Perl 5, cURL, módulos PECL	Node.js, Java (JSP/Tomcat), ASP.NET (todas las versiones), Zend Framework, PHP 5 y 7.
Bases de Datos	Soportadas: MySQL 5.7 (CentOS) o MySQL 8.0 (AlmaLinux), MySQLi, PDO (pdo_mysql)	PostgreSQL, MariaDB, Microsoft SQL Server, Oracle, MongoDB.
CMS y E-commerce	Gestores: WordPress (y WP MU), Drupal, Joomla (hasta 5.5.2) E-commerce: Magento (2.3), PrestaShop (1.7 y 8), OpenCart E-learning: Moodle (hasta 4.1)	Versiones más recientes o no especificadas de Joomla y Moodle requerirán VPS.

Categoría	Herramientas y Tecnologías Compatibles	Incompatibilidades Notables
Paneles y Servicios	Paneles: cPanel, WHM, Softaculous (instalador de apps) Accesos: FTP (Cyberduck), SSH2, WebDisk, Tareas Cron Correo: Webmail (RoundCube), IMAP, POP3, SMTP, Exim	Plesk, Docker, VPN, Servidores de juegos, Terminales como KVM o Remote Desktop.
Seguridad y SSL	Certificados: SSL Gratis (Let's Encrypt) para todos los dominios, Wildcard SSL, OpenSSL Antispam: SpamAssassin	SSL compartido, Certificación HIPAA compliance, PCI Compliance.
Otras Herramientas	Git (Cliente), SVN (Cliente), AWstats, ImageMagick, Archivos comprimidos (.zip, .tar.gz), Ajax, HTML	FFmpeg, Live streaming, Load balancing, Redis/Memcached, XAMPP.

Herramientas que pueden implementarse

I. Programación

- **Python:** el lenguaje de programación que tenemos contemplado usar para el desarrollo del backend del sistema gracias a su flexibilidad, compatibilidad con una variedad de bibliotecas criptográficas y su nivel de dominio por los miembros del equipo.
- **Django/Flask:** el primero “es un marco web de alto nivel en Python que fomenta el desarrollo rápido y el diseño limpio y pragmático” (Django, 2026). El segundo, “un marco de trabajo ligero para aplicaciones web WSGI(...) diseñado para facilitar y agilizar los primeros pasos, con la capacidad de ampliarse hasta aplicaciones complejas” (Flask, 2026). Ambos marcos podrían permitir construir aplicaciones seguras a través de arquitecturas cliente-servidor y facilitar la creación de APIs para que diferentes partes del sistema puedan comunicarse entre ellas.
- **FastAPI:** “un marco web moderno y rápido (de alto rendimiento) para crear API con Python basado en sugerencias de tipos estándar de Python” (FastAPI, 2025), que puede nos podría ser útil para conectar el backend del sistema con la base de datos y la interfaz web.

II. Criptografía

- **OpenSSL**: “Un kit de herramientas robusto, de calidad comercial y con todas las funciones para el protocolo Transport Layer Security (TLS)” (OpenSSL Project, 2023). Es decir, una biblioteca criptográfica comúnmente usada que nos permite implementar funciones como la generación de claves, certificados digitales, firmas electrónicas, etc.
- **PyCA Cryptography**: biblioteca para Python que permite implementar de forma segura algoritmos criptográficos modernos, como funciones hash, cifrado o incluso firmas digitales (Cryptography Project, 2023).
- **Hashlib**: “implementa una interfaz común para muchos algoritmos diferentes de hash seguro y resumen de mensajes” (Python Software Foundation, 2023). Será útil para verificar la integridad de nuestra información y detectar posibles modificaciones que no hayan sido autorizadas.

III. Bases de datos

- **MySQL**: como se explicó en la sección de “Aproximación tentativa”, es un sistema de gestión de bases de datos relacionales que podremos utilizar para la gestión de información (desde identidades hasta registros de auditoría).

IV. Organización del proyecto

- **ProjectLibre**: herramienta de gestión de proyectos que usaremos para planificar y monitorear actividades, permitiendo el correcto trabajo entre los integrantes del equipo.

V. Gestión de archivos y control de versiones

- **Git**: como fue explicado anteriormente, es un sistema de control de versiones que permite gestionar cambios han sido hechos en el código fuente, facilitando así el trabajo colaborativo.
- **Github**: al igual que Git, fue explicado en mayor detalle en la sección anterior. En general, es una plataforma basada en Git que permite a los integrantes del equipo colaborar en proyectos, crear repositorios de código y mantener un historial de modificaciones realizadas de manera ordenada y clara.

Uso de herramientas en modo exploratorio:

En primer lugar, demostraremos la certificación y la comunicación segura mediante TLS v1.3 (Transport Layer Security). Para ello, podemos usar las bibliotecas *PyCA/cryptography* y *OpenSSL* (como se mencionó anteriormente).

En nuestro ejemplo (que está disponible en el repositorio de GitHub en *examples/tls*), usaremos un certificado autofirmado, pero en un escenario real este sería firmado por una Autoridad Certificadora (CA).

El archivo *generate_certificate.py* genera un par de claves RSA:

```
key = rsa.generate_private_key(
    public_exponent=65537, # 2^16 + 1, Fermat prime and efficient for computing
    key_size=4096          # Double the minimum key size
)
```

Código 1. Creación de un par de claves (pública y privada)

Usamos el número 65537 como exponente público. En primer lugar, es un número primo y, en segundo lugar, es un primo de Fermat (es decir, de la forma $2^{2^n} + 1$), lo que facilita los cálculos. Además, usamos el tamaño de 4096 bits, que es el doble del tamaño mínimo.

La clave privada se guarda en el archivo *private_key.pem*. A continuación, se crea un certificado con los detalles de la organización y se firma con la clave pública:

```
subject = x509.Name([
    x509.NameAttribute(NameOID.COUNTRY_NAME, "MX"),
    x509.NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, "Nuevo León"),
    x509.NameAttribute(NameOID.LOCALITY_NAME, "Monterrey"),
    x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Example Company"),
    x509.NameAttribute(NameOID.COMMON_NAME, "example-company.com")
])

# Create certificate
cert = x509.CertificateBuilder().\
    subject_name(subject).\
    issuer_name(subject).\
    public_key(key.public_key()).\
    serial_number(x509.random_serial_number()).\
    not_valid_before(datetime.datetime.now()).\
    not_valid_after(datetime.datetime.now() + datetime.timedelta(weeks=4)).\
    add_extension(x509.SubjectAlternativeName([x509.DNSName(HOST)]),
```

```
critical=False).\
sign(key, hashes.SHA256())
```

Código 2. Creación de un certificado

Además, se definen formalidades como el número de serie y el período de validez. Todo esto se realiza mediante el submódulo *x509* de la biblioteca *PyCA/cryptography*, que se explicará con más detalle a continuación.

Al ejecutar *server.py*, la biblioteca *ssl* se utiliza de la siguiente manera:

```
ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER) # Act as server
ctx.minimum_version = ssl.TLSVersion.TLSv1_3 # Require at least TLSv1.3 to be used

ctx.load_cert_chain(certfile="./certificate.pem", keyfile="./private_key.pem",
                    password=pwd)
```

Código 3. Creación de un contexto de TLSv1.3 para un servidor

Aquí, el servidor se configura para usar el archivo de certificado y carga la clave privada correspondiente desde otro archivo.

Luego, con este contexto, se puede establecer fácilmente una conexión con un cliente:

```
with socket.socket() as sock:
    sock.bind((HOST, PORT)) # Listen on HOST:PORT for TCP connections
    sock.listen()

    with ctx.wrap_socket(sock, server_side=True) as ssock: # Secure SOCKET
        conn, addr = ssock.accept() # Accept incoming connection

        with conn:
            data = conn.recv(1024) # Receive data (max. 1024 bytes)

            print("[*] Received:", data.decode()) # Show received data
            conn.send(b"Hello, Client!") # Send data
            conn.close() # Close connection
```

Código 4. Establecer una conexión utilizando TLSv1.3 en el lado del servidor

El cliente, por su parte, simplemente carga el archivo de certificado y configura el contexto para que se ejecute en modo cliente:

```
ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT) # Act as client
```

```
ctx.minimum_version = ssl.TLSVersion.TLSv1_3 # Require at least TLSv1.3 to be used
ctx.verify_mode = ssl.CERT_REQUIRED           # Require server to show certificate
ctx.load_verify_locations(cafile="./certificate.pem") # Only accepted certificate
```

Código 5. Creación de un contexto de TLSv1.3 para un cliente

Después, se establece una conexión con el servidor con la misma facilidad:

```
with socket.socket() as sock:
    with ctx.wrap_socket(sock, server_hostname=HOST) as ssock: # Secure SOCKET
        ssock.connect((HOST, PORT))                            # Connect via TCP to HOST:PORT

        ssock.sendall(b"Hello, Server!")                       # Send data
        data = ssock.recv(1024)                                # Receive data (max. 1024 bytes)
        print("[*] Received:", data.decode())                  # Show received data
        ssock.close()                                           # Close connection
```

Código 6. Establecer una conexión utilizando TLSv1.3 en el lado del cliente

Para realizar este ejemplo, solo necesita ejecutar primero *generate_certificate.py* con una contraseña como argumento de línea de comandos. Luego, puede ejecutar *server.py* con la misma contraseña como argumento de línea de comandos y ejecutar simultáneamente *client.py*. Entonces, la ejecución *server.py* debería mostrar el mensaje "Hello, Server!" y *client.py* debería mostrar el mensaje "Hello, Client!".

Biblioteca de gestión de certificados: cryptography.x509

Para la gestión del ciclo de vida de los certificados digitales dentro del sistema, se utilizará el submódulo **cryptography.x509**, parte de la biblioteca PyCA Cryptography ya descrita en la sección de herramientas. Este submódulo implementa el estándar X.509 definido en el RFC 5280, el cual es precisamente citado como base para las Listas de Revocación de Certificados (CRL) dentro del sistema de gestión de identidades.

El módulo **cryptography.x509** permite cubrir de forma nativa todo el ciclo de vida de una identidad digital dentro del sistema, que es la razón central por la que se optó por un gestor de identidades y no únicamente un sistema de autenticación simple.

Las operaciones que gestiona dentro del sistema son:

Alta: Mediante `x509.CertificateBuilder()`, `x509.Name()` y `x509.random_serial_number()`, se emite un certificado firmado por Casa Monarca como Autoridad Certificadora (CA) raíz, vinculando criptográficamente la identidad del usuario con su clave pública.

x509.CertificateBuilder(): Inicia la creación de un nuevo certificado, permitiendo configurar sus atributos.

x509.Name(): Define el "Nombre Distinguido" (DN), estableciendo la identidad del sujeto o emisor.

x509.random_serial_number(): Genera un número de serie único y aleatorio para el certificado.

Expiración automática: A través de `.not_valid_before()` y `.not_valid_after()`, cada certificado tiene un periodo de validez definido. Al vencerse, el sistema puede marcarlo automáticamente como inactivo en MySQL sin intervención del administrador.

Propuesta inicial

La propuesta inicial fue explicada en mayor detalle en la sección “Aproximación tentativa”. No obstante, nos pareció importante resumir, en este apartado, lo que se explicó anteriormente.

La propuesta consiste en desarrollar una aplicación web basada en arquitectura cliente-servidor que permita gestionar de forma segura la identidad digital y el acceso a la información dentro de Casa Monarca. En esta arquitectura, el backend del sistema se encargará de la lógica de seguridad y la gestión de identidades, mientras que el frontend proporcionará una interfaz sencilla pero eficaz para que cualquier colaborador o voluntario de Casa Monarca pueda interactuar con la plataforma.

Además, el sistema se basará en un modelo de verificación constante, en el que cada usuario debe validar su identidad antes de acceder a información sensible. De esta forma, los voluntarios y colaboradores sólo podrán consultar o modificar los datos que correspondan a su rol dentro de la organización.

Asimismo, el sistema también integrará el uso de certificados digitales y herramientas criptográficas para proteger la información almacenada y garantizar que los documentos no puedan ser alterados. Para lograr lo anterior, planeamos usar bibliotecas criptográficas y una base de datos que permita gestionar identidades, listas de revocación y registros de auditoría.

Con esta estructura se busca facilitar la gestión de usuarios, proteger los datos personales de los beneficiarios y mantener un control claro sobre quién accede a la información y con qué propósito.

Enlace al repositorio

https://github.com/A01831983/MA2006B_Reto

Cronograma de Gantt

Con el objetivo de garantizar el cumplimiento de las entregas del proyecto, se ha desarrollado un cronograma de actividades utilizando la herramienta *ProjectLibre*. Este esquema organiza las visitas al socio formador, periodos sin clases del curso de *Uso de álgebras modernas para la seguridad y criptografía* y entregas de las actividades propias a la resolución del reto con la Organización Socio Formadora, y busca monitorear el progreso individual y colectivo.

- **Semanas 1-3:** Sesiones de Metodología
- **Semana 4:** Introducción al reto
- **Intermedio:** Investigación por parte de todos los miembros del equipo y generación de ideas para la aproximación para resolver el reto.
- **Semana 5:** Sesión de preguntas y respuestas relacionadas al reto con la Organización Socio Formadora
- **Semana 7:** Consolidación de la propuesta de solución y exploración de las herramientas de trabajo
- **Semana 8:** Creación de los componentes de la solución
- **Semana 9:** Conexión de los componentes
- **Semana 10:** Pruebas de la solución
- **Semana 11:** Prueba final, documentación de evidencias y presentación de solución

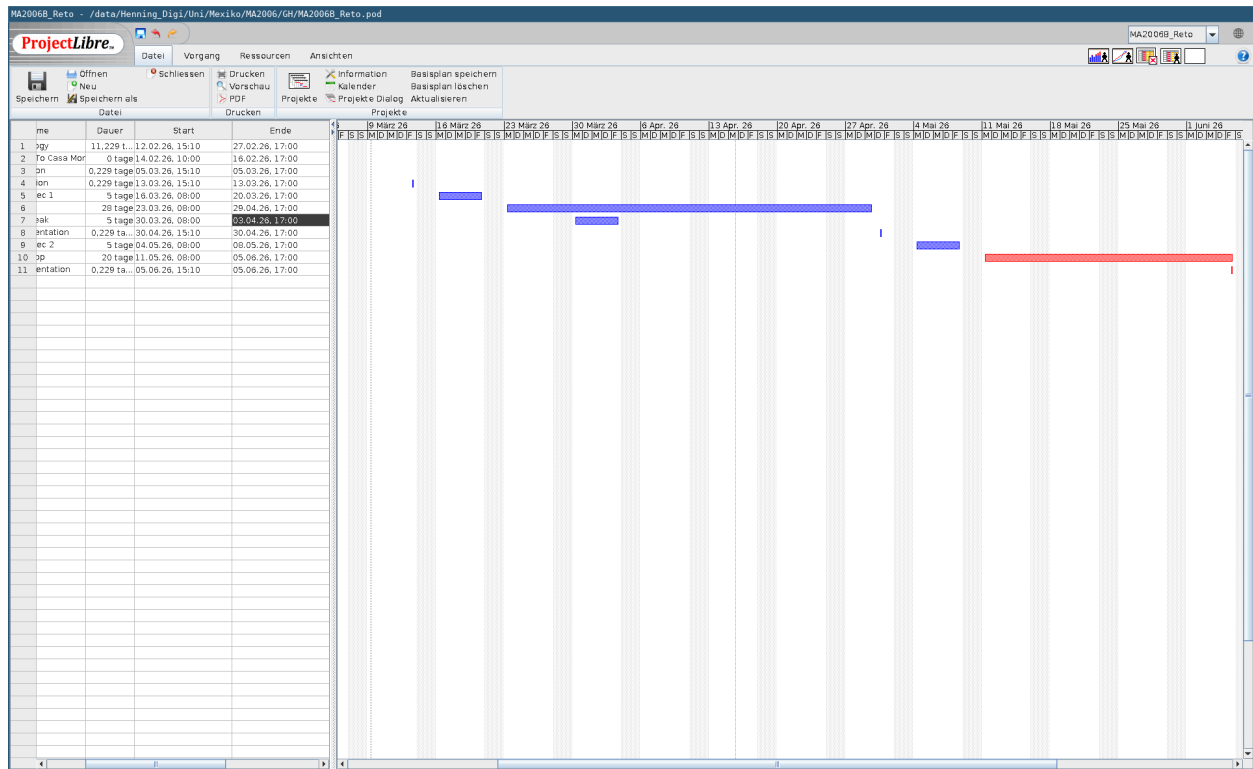


Figura 1. Cronograma en ProjectLibre

Preguntas formuladas por integrante en la sesión con Socio Formador

1. *Valeria:* ¿Cuentan en la actualidad con una base de datos de colaboradores y usuarios centralizada o los registros están dispersos en distintas áreas? ¿Qué niveles de prioridad tienen las distintas áreas de Casa Monarca (Humanitaria, Legal, Salud) en el acceso al sistema de identidades?
2. *Ximena:* Dado que el sistema debe ser fácil de usar, ¿cuál es el perfil tecnológico promedio de los voluntarios que rotan en la organización? ¿Cuáles son las características de la infraestructura que tienen actualmente?
3. *Henning:* ¿Qué tamaño debería tener el sistema? ¿Unas pocas docenas de usuarios, unos cientos o quizás unos miles?
4. *Fernando:* ¿Los colaboradores utilizan principalmente dispositivos personales, o existen equipos institucionales? ¿El acceso al sistema y el sistema debe poder funcionar sin internet constante?
5. *Diego:* ¿Quiénes en Casa Monarca deben tener el permiso para emitir, modificar o revocar las identidades de los migrantes dentro del sistema? ¿Con qué frecuencia cambian o se unen nuevos usuarios y/o voluntarios a Casa Monarca?
6. *Alberto:* ¿Cuáles son los formatos que utilizan para registrar a los migrantes que buscan apoyo? ¿El área de TI cuenta con personal dedicado o es cubierta por voluntarios con conocimientos variables? ¿Existen procesos donde agentes externos como proveedores o el gobierno ya interactúa digitalmente con ustedes?

Referencias

- Chacon, S., & Straub, B. (2014). *Pro Git (2da ed)*. Apress. <https://git-scm.com/book/en/v2>
- Chaparro Zuñiga, H. D., & Hecht, P. (2023). *Análisis de arquitectura y diseño de metodología de seguridad de confianza cero para los servicios en la nube*. Universidad de Buenos Aires. http://bibliotecadigital.econ.uba.ar/download/tpos/1502-2963_ChaparroZ.pdf
- Cooper, D., Santesson, S., Farrell, S., Boeyen, S., Housley, R., & Polk, W. (2008). *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile (RFC 5280)*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/pdf/rfc5280.txt.pdf>
- Cryptography. (2026).
- *Welcome to pyca/cryptography*. <https://cryptography.io/en/latest/>
 - *X.509 Tutorial*. <https://cryptography.io/en/latest/x509/tutorial/#creating-a-self-signed-certificate>
- Django. (2026). *Django Project*. <https://www.djangoproject.com/>
- FastAPI. (2025). *Tiangolo.com*. <https://fastapi.tiangolo.com/>
- Github. (2026). *Why Choose GitHub*. <https://github.com/why-github>
- Grassi, P., Garcia, M., & Fenton, J. (2017). *Digital Identity Guidelines (NIST SP 800-63-3)*. National Institute of Standards and Technology. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-63-3.pdf>
- Mohammed, K. I., Shanmugam, B., & El-Den, J. (2025). *Evolution of DevSecOps and Its Influence on Application Security: A Systematic Literature Review*. Technologies, 13(12), 548. <https://doi.org/10.3390/technologies13120548>
- Nadipalli, R. (2023). *Cloud-Native DevSecOps: A Framework for Secure Continuous Delivery*. International Journal of Computing and Engineering, 3(2), 1–9. <https://doi.org/10.47941/ijce.3104>
- OpenSSL Project. (2023). *OpenSSL: Cryptography and SSL/TLS Toolkit*. <https://www.openssl.org>
- Oracle. (2024). *MySQL 8.4 Reference Manual*. MySQL Documentation

PostgreSQL Global Development Group. (2023). *PostgreSQL Documentation*.
<https://www.postgresql.org/docs/>

Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice (7ma ed.)*. Pearson.
<https://faculty.ksu.edu.sa/sites/default/files/cryptography-network-security-5th-edition.pdf>

Flask. *Flask Documentation (3.1.x)*. (2026). Palletsprojects.com.
<https://flask.palletsprojects.com/en/stable/>

Zong, C. (2025). *The Mathematical Foundation of Post-Quantum Cryptography*. Research (Washington, D.C.), 8, 0801. <https://doi.org/10.34133/research.0801>